

Lab4. Stored subprograms and triggers

Part 1:

1. Write PL/SQL code to create a function to check if a number is prime.

--Code

```
DECLARE
    l_check_num Number := 5;
    l_check Number := 1;
BEGIN
    FOR n_index IN 2 .. (l_check_num - 1)
    LOOP
        IF MOD(l_check_num, n_index) = 0 THEN
            l_check := 0;
            EXIT;
        END IF;
    END LOOP;

    IF l_check = 0 then
        DBMS_OUTPUT.PUT_LINE('Number is not prime');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Number is prime');
    END IF;
END;
```

```
SQL> DECLARE
  2  l_check_num Number := 5;
  3  l_check Number := 1;
  4  BEGIN
  5  FOR n_index IN 2 .. (l_check_num - 1)
  6  LOOP
  7  IF MOD(l_check_num, n_index) = 0 THEN
  8  l_check := 0;
  9  EXIT;
10  END IF;
11  END LOOP;
12
13  IF l_check = 0 then
14  DBMS_OUTPUT.PUT_LINE('Number is not prime');
15  ELSE
16  DBMS_OUTPUT.PUT_LINE('Number is prime');
17  END IF;
18
19  END;
20  /
Number is prime

PL/SQL procedure successfully completed.
```

```

SQL> DECLARE
  2  l_check_num Number := 8;
  3  l_check Number := 1;
  4  BEGIN
  5    FOR n_index IN 2 .. (l_check_num - 1)
  6    LOOP
  7      IF MOD(l_check_num, n_index ) = 0 THEN
  8        l_check :=0;
  9        EXIT;
 10      END IF;
 11    END LOOP;
 12
 13    IF l_check = 0 then
 14      DBMS_OUTPUT.PUT_LINE('Number is not prime');
 15    ELSE
 16      DBMS_OUTPUT.PUT_LINE('Number is prime');
 17    END IF;
 18
 19  END;
 20  /
Number is not prime

PL/SQL procedure successfully completed.

```

2. Write PL/SQL code to create a procedure to calculate the factorial of a number.

--Code

DECLARE

 l_fac_num Number := 9;

 n_index Number := 2;

BEGIN

 LOOP

 IF l_fac_num = n_index THEN

 DBMS_OUTPUT.PUT_LINE(n_index);

 EXIT;

 ELSIF MOD(l_fac_num ,n_index) = 0 THEN

 DBMS_OUTPUT.PUT_LINE(n_index);

 l_fac_num := l_fac_num / n_index;

 ELSE n_index := n_index + 1;

 END IF;

 END LOOP;

END;

```

SQL> DECLARE
  2  l_fac_num Number := 9;
  3  n_index Number := 2;
  4  BEGIN
  5  LOOP
  6  IF l_fac_num = n_index THEN
  7  DBMS_OUTPUT.PUT_LINE(n_index );
  8  EXIT;
  9  ELSIF MOD( l_fac_num ,n_index ) = 0 THEN
 10  DBMS_OUTPUT.PUT_LINE(n_index );
 11  l_fac_num := l_fac_num / n_index;
 12  ELSE n_index := n_index + 1;
 13  END IF;
 14  END LOOP;
 15
 16
 17 END;
 18 /
3
3

PL/SQL procedure successfully completed.

```

```

SQL> DECLARE
  2  l_fac_num Number := 8;
  3  n_index Number := 2;
  4  BEGIN
  5  LOOP
  6  IF l_fac_num = n_index THEN
  7  DBMS_OUTPUT.PUT_LINE(n_index );
  8  EXIT;
  9  ELSIF MOD( l_fac_num ,n_index ) = 0 THEN
 10  DBMS_OUTPUT.PUT_LINE(n_index );
 11  l_fac_num := l_fac_num / n_index;
 12  ELSE n_index := n_index + 1;
 13  END IF;
 14  END LOOP;
 15
 16
 17 END;
 18 /
2
2
2
2

PL/SQL procedure successfully completed.

```

3. Write a PL/SQL function to calculate the bonus amount for employees based on their salary and years of service.

--Code

```

CREATE OR REPLACE FUNCTION calculate_bonus(salary INTEGER, years_of_service
INTEGER) RETURN FLOAT IS
  BEGIN

```

```

        RETURN salary * (years_of_service * 0.1);
    END calculate_bonus;

-- Test
DECLARE
    v_bonus FLOAT;
BEGIN
    -- Loop through all employees
    FOR emp IN (SELECT employee_id, salary, hire_date FROM employees) LOOP
        v_bonus := calculate_bonus(emp.salary, extract(year from sysdate) - extract(year from
emp.hire_date + 1));

        DBMS_OUTPUT.PUT_LINE('Bonus for Employee ' || emp.employee_id || ': ' ||
v_bonus);
    END LOOP;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

The screenshot shows the Oracle SQL Developer interface. The top window, titled 'Query Builder', contains a PL/SQL script. The script defines a function 'calculate_bonus' and a block to calculate bonuses for all employees. The bottom window, titled 'Script Output', shows the execution results, including a confirmation that the function was compiled and a list of bonus values for 13 employees.

```

CREATE OR REPLACE FUNCTION calculate_bonus(salary INTEGER, years_of_service INTEGER) RETURN FLOAT IS
BEGIN
    RETURN salary * (years_of_service * 0.1);
END calculate_bonus;

DECLARE
    v_bonus FLOAT;
BEGIN
    -- Loop through all employees
    FOR emp IN (SELECT employee_id, salary, hire_date FROM employees) LOOP
        -- Calculate the number of years of service
        v_bonus := calculate_bonus(emp.salary, extract(year from sysdate) - extract(year from emp.hire_date + 1));
        -- Output the bonus for each employee
        DBMS_OUTPUT.PUT_LINE('Bonus for Employee ' || emp.employee_id || ': ' || v_bonus);
    END LOOP;

    EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;

```

Script Output x

Task completed in 0.033 seconds

```

Function CALCULATE_BONUS compiled

Bonus for Employee 100: 50400
Bonus for Employee 101: 32300
Bonus for Employee 102: 39100
Bonus for Employee 103: 16200
Bonus for Employee 104: 10200
Bonus for Employee 105: 9120
Bonus for Employee 106: 8640
Bonus for Employee 107: 7140
Bonus for Employee 108: 26417.6
Bonus for Employee 109: 19800
Bonus for Employee 110: 15580
Bonus for Employee 111: 14630
Bonus for Employee 112: 14040
Bonus for Employee 113: 11730

```

4. Write a PL/SQL that creates a procedure to calculate the average salary of employees in a specific department.

--Code

```
CREATE OR REPLACE PROCEDURE average_salary(department_id INTEGER,
avg_salary OUT NUMBER) IS
```

```
BEGIN
```

```
    SELECT AVG(salary) INTO avg_salary
    FROM employees
    WHERE department_id = department_id;
```

```
    IF avg_salary IS NULL THEN
```

```
        avg_salary := 0;
```

```
    END IF;
```

```
END average_salary;
```

-- Test

```

DECLARE
    avg_salary NUMBER;
    dep_id NUMBER := 90; -- the department ID you want to check
BEGIN
    average_salary(dep_id, avg_salary);
    DBMS_OUTPUT.PUT_LINE('Average Salary for Department ' || dep_id || ': ' ||
avg_salary);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;

```

The screenshot displays the SQL Developer interface. The top toolbar shows a timer at 0.027 seconds. The 'Query Builder' tab is active, showing the following SQL code:

```

CREATE OR REPLACE PROCEDURE average_salary(department_id INTEGER, avg_salary OUT NUMBER) IS
BEGIN
    SELECT AVG(salary) INTO avg_salary
    FROM employees
    WHERE department_id = department_id;

    IF avg_salary IS NULL THEN
        avg_salary := 0;
    END IF;
END average_salary;

DECLARE
    avg_salary NUMBER;
    dep_id NUMBER := 90; -- the department ID you want to check
BEGIN
    -- Call the procedure to calculate the average salary
    average_salary(dep_id, avg_salary);

    -- Print the average salary
    DBMS_OUTPUT.PUT_LINE('Average Salary for Department ' || dep_id || ': ' || avg_salary);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;

```

The 'Script Output' window at the bottom shows the execution results:

```

Procedure AVERAGE_SALARY compiled
Average Salary for Department 90: 6456.754716981132075471698113207547169811
PL/SQL procedure successfully completed.

```

5. Write a PL/SQL that creates a function to retrieve the total number of employees in a specific job title.

--Code

```

CREATE OR REPLACE FUNCTION count_employees_by_job(job_title IN VARCHAR2)
RETURN INTEGER IS
    v_count INTEGER;
BEGIN

    SELECT COUNT(*) INTO v_count
    FROM employees
    WHERE job_id = job_title;

    RETURN v_count;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RETURN -1;
END count_employees_by_job;

-- Test
DECLARE
    emp_count INTEGER;
    job_title VARCHAR2(25) := 'SA_REP'; --Enter job_title you want to check
BEGIN
    emp_count := count_employees_by_job(job_title);
    DBMS_OUTPUT.PUT_LINE('Total Employees with job title ' || job_title || ': ' ||
emp_count);
END;

```

The screenshot displays the Oracle SQL Developer interface. The top pane, titled 'Query Builder', contains a PL/SQL script. The script defines a function `count_employees_by_job` that takes a `job_title` as input and returns the count of employees for that job title. It includes an exception handler for errors. Below the function, a test block is provided, which declares a variable `emp_count` and a `job_title` set to 'SA_REP'. It then calls the function and prints the result using `DBMS_OUTPUT.PUT_LINE`. The bottom pane, titled 'Script Output', shows the execution results: the function was compiled successfully, the output for 'SA_REP' is 30, and the overall PL/SQL procedure completed successfully.

```
CREATE OR REPLACE FUNCTION count_employees_by_job(job_title IN VARCHAR2)
RETURN INTEGER IS
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM employees
    WHERE job_id = job_title;

    RETURN v_count;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RETURN -1;
END count_employees_by_job;
-- Test
DECLARE
    emp_count INTEGER;
    job_title VARCHAR2(25) := 'SA_REP'; --Enter job_title you want to check
BEGIN
    emp_count := count_employees_by_job(job_title);
    DBMS_OUTPUT.PUT_LINE('Total Employees with job title ' || job_title || ': ' || emp_count);
END;
```

Script Output x

Task completed in 0.033 seconds

Function COUNT_EMPLOYEES_BY_JOB compiled

Total Employees with job title SA_REP: 30

PL/SQL procedure successfully completed.

6. Write a PL/SQL that creates a procedure to update an employee's salary based on their performance rating.

-- Em không tìm thấy performance rating trong table hr thầy ơi???